МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № 3 по курсу «Операционные системы и архитектура компьютера»

Выполнил: Федосеев Николай Владимирович
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 22.10.2026
Оценка: _____

Москва, 2026

Условие

Цель работы: получить практические навыки управления потоками в ОС и синхронизации между ними.

Задание, вариант 9.

Рассчитать детерминант матрицы, используя определение детерминанта.

Программа должна быть написана на языке Си и использовать стандартные средства создания потоков ОС. Максимальное количество одновременно работающих потоков задаётся ключом запуска. Необходимо показать количество потоков средствами ОС, исследовать ускорение и эффективность алгоритма при изменении входных данных и числа потоков, объяснить результаты.

Метод решения

Определитель вычисляется по определению: перебираются перестановки столбцов, для каждой берётся по одному элементу из каждой строки и столбца. Их произведение прибавляется или вычитается в зависимости от чётности перестановки:

$$\det A = \sum_{\sigma \in S_n} (-1)^{\mathrm{inv}(\sigma)} \prod_{i=0}^{n-1} a_{i,\sigma(i)}.$$

Здесь $\mathrm{inv}(\sigma)$ - число инверсий: пар, в которых левый элемент перестановки больше правого. Всего имеется $n!$ слагаемых. Работа прямого перебора имеет порядок $O(n \cdot n!)$; потоки распределяют эту работу, но не устраняют факториальный рост.

При $n \geq 2$ вычисление разбивается на задачи (`job` в коде). Каждая задача перебирает перестановки с фиксированными первыми двумя элементами. Всего таких задач $n(n-1)$, каждая перебирает $(n-2)!$ вариантов хвоста. При $n = 1$ задача одна. Функция `prepare_permutation` заполняет выбранное начало и отсортированный хвост, затем определяет начальный знак по инверсиям. `permutation_next` переставляет хвост и переключает знак при каждом обмене.

Исполнитель с номером i получает задания i, $i + p$, $i + 2p$ и так далее, где p - фактическое число исполнителей. Главный поток тоже считает: создаются $p - 1$ дополнительных потоков. Число исполнителей ограничивается числом заданий, но не количеством ядер.

Матрица общая и во время расчёта не меняется. У каждого исполнителя свой массив перестановки. Сумма задания записывается в отдельную ячейку `sums[job]`, которую другие исполнители не изменяют. После завершения потоков главный складывает ячейки в порядке номеров заданий. Общая очередь и мьютекс для вычислений не нужны: задачи распределены заранее, а ожидание и получение результатов выполняются через `pthread_join`.

Описание программы

Структура файлов

Таблица 1 – Основные файлы проекта

Файл	Назначение
`src/main.c`	Аргументы, открытие и закрытие файла, замер времени и вывод.
`src/matrix.h/.c`	Структура матрицы, статусы, выделение и освобождение памяти.
`src/matrix_txt.h/.c`	Чтение матрицы из текстового потока.
`src/permutation.h/.c`	Следующая перестановка и обновление знака.
`src/determinant.h/.c`	Разбиение на задания, рабочая функция, запуск и ожидание потоков, сбор результата.
`src/generate.py`	Простой генератор TXT: размер и n^2 случайных чисел.
`src/CMakeLists.txt`	Сборка программы C17 с POSIX threads.

Основные типы данных и функции

`struct Matrix` хранит размер `size` и указатель `data` на непрерывный массив `int64_t`. Элемент строки `row` и столбца `column` имеет индекс `row * size + column`. Размеры и индексы имеют тип `size_t`.

`matrix_init` выделяет память пустой матрице, `matrix_destroy` освобождает её и обнуляет поля. `matrix_read_txt` читает размер и элементы во временную матрицу, а наружу передаёт её только после успешного чтения. Открытый поток принадлежит `main`.

`struct Worker` содержит адрес матрицы, номер исполнителя, количество исполнителей и заданий, общий массив сумм и признак успеха. `worker_run(void *argument)` получает адрес этой структуры, выделяет собственную перестановку и выполняет свои задания.

`determinant_compute` выделяет общие рабочие массивы, подготавливает контексты, создаёт дополнительные потоки, выполняет работу исполнителя 0 и присоединяет созданные потоки. Возвращает `true` или `false`. `struct Calculation` получает определитель и фактическое число исполнителей только при успехе.

Потоки и системные вызовы

Таблица 2 – Основные операции программы

Функция в коде	Назначение
`pthread_create`	Запускает `worker_run` в новом потоке процесса.
`pthread_join`	Ожидает завершения конкретного рабочего потока.
`clock_gettime`	Получает время `CLOCK_MONOTONIC` до и после расчёта.
`fopen`, `fscanf`, `fclose`	Открывают, читают и закрывают текстовый файл.
`malloc`, `free`	Выделяют и освобождают матрицу и рабочие массивы.
`printf`, `fflush`	Выводят результат и проверяют сброс буфера вывода.

Ключевые фрагменты кода

Каждый исполнитель получает непересекающиеся номера заданий.

Листинг 1 – Распределение заданий (determinant.c)

```
    for (size_t job = worker->index; job < worker->job_count &&
    ↪  worker->success; job += worker->thread_count) {
        bool negative = prepare_permutation(permutation, size, job);
        int64_t sum = 0;
```

Перед запуском потоков все контексты уже заполнены. При успешном `pthread_create` счётчик `created` увеличивается. Затем главный поток сам вызывает `worker_run` для контекста 0. При ошибке создания этот вызов пропускается, но уже запущенные потоки всё равно присоединяются.

Листинг 2 – Запуск дополнительных потоков (determinant.c)

```
        size_t created = 0;

        for (size_t index = 1; index < count; index++) {
            int code = pthread_create(&threads[created], NULL, worker_run,
            ↪  &workers[index]);

            if (code != 0) {
                fprintf(stderr, "pthread_create: %s\n", strerror(code));
                success = false;
                break;
            }

            created++;
        }

        if (success) {
            worker_run(&workers[0]);
        }
```

В произведениях и суммах используется `int64_t`. Каждое умножение, сложение и вычитание проверяется через `__builtin_*_overflow`.

Обработка ошибок

Проверяются аргументы запуска, открытие, чтение и закрытие файла, размеры выделений, результат `malloc`, ошибки потоков, арифметика, получение времени и вывод. Размеры выделений проверяются до умножения. При частичном отказе `malloc` успешно выделенные массивы освобождаются в общем месте очистки.

При ошибке `pthread_create` программа дожидается только действительно созданных потоков и возвращает неуспех. При арифметической ошибке исполнитель завершает свою работу, остальные заканчивают свои задания; итоговый определитель не публикуется. При ошибке `pthread_join` процесс завершается через `_Exit`: освобождать память, которой ещё может пользоваться поток, небезопасно.

Загрузчик рассчитан на подготовленный числовой ввод: значения должны помещаться в `int64_t`. `fscanf` здесь не обеспечивает безопасную проверку произвольно больших числовых записей. Переполнение промежуточного произведения или суммы считается ошибкой, даже если после математических сокращений итог мог бы поместиться.

Сборка и запуск

Используется CMake не ниже 4.3, стандарт C17. Макрос `_POSIX_C_SOURCE=200809L` открывает объявления POSIX-функций. Потоки подключаются через `Threads::Threads`.

```
cmake -S src -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build --parallel
./build/determinant --threads 3 src/examples/matrix.txt
```

Файл содержит положительный размер, затем ровно n^2 элементов. Для матрицы ниже определитель равен -3.

```
3
1 2 3
4 5 6
7 8 10
```

Флаг `threads` задаёт предел исполнителей вместе с главным потоком. Программа выводит определитель, фактическое число исполнителей и время. Замер `CLOCK_MONOTONIC` охватывает весь `determinant_compute`: выделение рабочей памяти, создание потоков, вычисления, ожидание, сложение сумм и очистку. Чтение файла и вывод исключены из времени подсчёта.

Демонстрация работы strace

Программа запущена из корня проекта командой:

```
strace -f -yy -s 256 -o report/inc/strace-3x3.txt ./build/determinant --threads 3
↪  src/examples/matrix.txt
```

`-f` включает рабочие потоки, `-yy` подписывает дескрипторы, `-s 256` увеличивает лимит отображаемых строковых данных, `-o` сохраняет трассу в файл. Полная трейс приведена в приложении Б. Номера строк ниже относятся к этому файлу.

Чтение матрицы

Загрузка библиотек занимает начало трассы. В строке 32 `openat` открывает `src/examples/matrix.txt` и возвращает дескриптор 3. На строке 34 `read` получает все 21 байт файла: размер и девять элементов. На строке 35 чтение возвращает 0, то есть EOF. На строке 36 дескриптор закрывается. Несколько вызовов `fscanf` используют буфер libc, поэтому отдельного `read` для каждого числа нет.

Создание потоков

Главный поток имеет TID 508983. В строках 42 и 47 два успешных `clone3` создают потоки с TID 508984 и 508985. Вместе с главным это три исполнителя, как задано ключом запуска.

Листинг 3 – Первый рабочий поток: системный вызов clone3

```
508983 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLON┘
↪  E_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
↪  child_tid=0x7fd4cb87dce8, parent_tid=0x7fd4cb87d990, exit_signal=0,
↪  stack=0x7fd4cb07d000, stack_size=0x7fff80, tls=0x7fd4cb87d6c0} =>
↪  {parent_tid=[508984]}, 88) = 508984
```

Флаг CLONE_VM означает общую память, CLONE_FILES - общую таблицу дескрипторов, CLONE_THREAD - принадлежность одному процессу. Поэтому это потоки одного процесса, а не три отдельных процесса. CLONE_SETTLS задаёт локальное хранилище потока. Перед созданием каждого рабочего потока библиотека выделяет память под его стек через mmap с флагом MAP_STACK. Затем вызовом madvise с параметром MADV_GUARD_INSTALL первые 4096 байт выделенной области отмечаются как защитные: обращение к ним вызывает SIGSEGV, что помогает обнаружить выход за границу стека. Эти действия видны в строках 39–40 и 44–45.

Ожидание и завершение

На строке 49 главный поток блокируется в futex; адрес совпадает с child_tid первого clone3. После завершения рабочих потоков вызов возвращает 0 на строке 68.

Листинг 4 – Начало ожидания и возврат из futex

```
508983 futex(0x7fd4cb87dce8, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 2, NULL,
↪  FUTEX_BITSET_MATCH_ANY <unfinished ...>
```

```
508983 <... futex resumed>)             = 0
```

В коде нет мьютекса: futex используется библиотекой потоков при ожидании завершения. Второй поток уже завершился к проверке его join, поэтому отдельного ожидания в трейсе не видно. Пометки unfinished и resumed - части записи одного вызова, между которыми в лог попали события других потоков.

Рабочие потоки завершаются через exit(0) на строках 61 и 67. Затем главный выводит результат:

```
Determinant: -3
Threads: 3
Time: 0.003735860 s
```

Это видно по трём write на строках 72–74. На строке 75 exit_group(0) завершает процесс.

Исследование ускорения и эффективности

Условия измерений

Использована одна матрица 13×13 из src/examples/matrix-13.txt. Во всех девяти запусках получен одинаковый определитель 7736. Изменялось только число исполнителей: 1, 2, 3, 4, 5, 6, 7, 8 и 12. Для каждого значения передан один замер: среднее и медиана не вычислялись. Исходный вывод сохранён в приложении B.

Процессор - AMD Ryzen 5 7535HS: 6 физических ядер, 12 логических CPU, по два аппаратных потока на ядро (SMT). Система - Fedora Linux 44, архитектура x86-64. Использован каталог build; его конфигурация на момент подготовки отчёта - Debug, флаги -g без -O3. Запуски бенчмарков выполнены без strace.

Для $n = 13$ имеется $13! = 6\,227\,020\,800$ слагаемых, распределённых по $13 \cdot 12 = 156$ заданиям. В каждом задании $11! = 39\,916\,800$ перестановок. По сравнению с шестью слагаемыми матрицы 3×3 это объясняет большой рост объёма работы; отдельные замеры разных размеров без трассировки не предоставлены.

Результаты

Ускорение и эффективность рассчитаны относительно однопоточного запуска:

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p} \cdot 100\%, \qquad T_1 = 244.303998558 \text{ s}.$$

Здесь p включает главный поток. Эффективность показывает отношение ускорения к числу исполнителей.

Таблица 3 – Время, ускорение и эффективность для матрицы 13×13

Потоков	Время, с	Ускорение	Эффективность, %
1	244.303998558	1.000	100.00
2	123.723745589	1.975	98.73
3	85.636897709	2.853	95.09
4	62.173046865	3.929	98.24
5	52.815789465	4.626	92.51
6	43.158549481	5.661	94.34
7	53.069405205	4.603	65.76
8	45.312672544	5.392	67.39
12	41.237441684	5.924	49.37

Разбор результатов

При переходе от одного к четырём исполнителям время уменьшилось с 244.304 до 62.173 с, ускорение составило 3.929. Для шести исполнителей получено 43.159 с и ускорение 5.661. В этих запусках рост числа исполнителей близок к использованию отдельных физических ядер.

Минимальное время среди переданных замеров - 41.237 с при 12 исполнителях, ускорение 5.924. По сравнению с шестью исполнителями время уменьшилось только примерно на 4.45%. Двенадцать логических CPU не равны двенадцати независимым ядрам: два аппаратных потока одного ядра разделяют его ресурсы. Поэтому эффективность при 12 исполнителях составляет 49.37%, хотя ускорение относительно одного потока почти шестикратное.

Зависимость не монотонна: семь исполнителей дали 53.069 с, восемь - 45.313 с, то есть оба запуска медленнее шестипоточного. Возможные причины - размещение потоков по физическим ядрам и SMT, фоновые задачи, частоты и нагрев процессора. Также задания неделимы: 156 делится на 6 и 12 без остатка, а при 7 и 8 исполнителях число заданий на исполнителя различается.

Числа 7, 8 и 12 превышают количество физических ядер, но не число логических CPU моего ноутбука. Превышение 12 логических CPU в этих измерениях не исследовалось. При дальнейшем увеличении числа готовых к работе потоков планировщику пришлось бы делить процессорное время между ними, так как дополнительные потоки сами по себе не создают вычислительных ресурсов.

Выводы

Реализовано вычисление определителя по определению с распределением перестановок между потоками. Матрица читается из TXT, число исполнителей задаётся ключом запус-

ка. Результаты собираются после завершения потоков. Проверяются ошибки выделений, потоков и арифметики. Трейс показывает создание двух рабочих потоков, ожидание и вывод ответа.

Для матрицы 13×13 все запуски дали определитель 7736. В переданных измерениях лучший результат получен при 12 исполнителях: 41.237 с вместо 244.304 с, ускорение 5.924. Основной выигрыш уже достигается на шести исполнителях, а увеличение до двенадцати даёт небольшой дополнительный эффект. Потоки ускоряют перебор, но факториальная сложность остаётся.

Источники

1. Linux man-pages: pthread_create(3).
2. Linux man-pages: pthread_join(3).
3. Linux man-pages: clone(2), clone3.
4. Linux man-pages: futex(2).
5. Linux man-pages: clock_gettime(2).
6. Руководство strace(1).

Приложение А. Исходный код программы и сборка

Листинг А.1 – Файл src/main.c

```
#include "determinant.h"
#include "matrix_txt.h"

#include <errno.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

static bool parse_threads(const char *text, size_t *count) {
    if (*text < '0' || *text > '9') {
        return false;
    }
    errno = 0;
    char *end = NULL;
    unsigned long value = strtoul(text, &end, 10);
    if (errno == ERANGE || *end != '\0' || value == 0) {
        return false;
    }
    *count = (size_t) value;
    return true;
}

int main(int argc, char *argv[]) {
    size_t thread_count;
    if (argc != 4 || strcmp(argv[1], "--threads") != 0 ||
    ↪  !parse_threads(argv[2], &thread_count)) {
        fprintf(stderr, "Usage: %s --threads N matrix.txt\n", argv[0]);
        return EXIT_FAILURE;
    }

    FILE *input = fopen(argv[3], "r");
    if (input == NULL) {
        perror("Cannot open matrix file");
        return EXIT_FAILURE;
    }

    struct Matrix matrix = {0};
    enum MatrixStatus status = matrix_read_txt(input, &matrix);
    int close_result = fclose(input);

    bool success = status == MATRIX_OK && close_result == 0;
    if (close_result == EOF) {
        perror("Cannot close matrix file");
    }
    if (status != MATRIX_OK) {
        fprintf(stderr, "Cannot read matrix: %s\n",
        ↪  matrix_status_string(status));
    }
```

```
    struct timespec start = {0};
    struct timespec finish = {0};
    struct Calculation calculation = {0};

    if (success && clock_gettime(CLOCK_MONOTONIC, &start) == -1) {
        perror("clock_gettime(start)");
        success = false;
    }

    if (success) {
        success = determinant_compute(&matrix, thread_count, &calculation);
    }

    if (success && clock_gettime(CLOCK_MONOTONIC, &finish) == -1) {
        perror("clock_gettime(finish)");
        success = false;
    }

    if (success) {
        double elapsed = (double) (finish.tv_sec - start.tv_sec) + (double)
        ↪  (finish.tv_nsec - start.tv_nsec) / 1e9;

        if (printf("Determinant: %ld\nThreads: %zu\nTime: %.9f s\n",
        ↪  calculation.determinant, calculation.threads,
                   elapsed) < 0 || fflush(stdout) == EOF) {
            perror("Cannot write result");
            success = false;
        }
    }

    matrix_destroy(&matrix);

    return success ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Листинг A.2 – Файл src/matrix.h

```
#ifndef LABA3_MATRIX_H
#define LABA3_MATRIX_H

#include <stddef.h>
#include <stdint.h>

struct Matrix {
    size_t size;
    int64_t *data;
};

enum MatrixStatus {
    MATRIX_OK = 0,
    MATRIX_INVALID_ARGUMENT,
    MATRIX_INVALID_SIZE,
```

```
    MATRIX_SIZE_OVERFLOW,
    MATRIX_NO_MEMORY,
    MATRIX_INVALID_ELEMENT,
    MATRIX_MISSING_ELEMENT,
    MATRIX_EXTRA_DATA,
    MATRIX_READ_ERROR
};

// Requires an empty matrix initialized with {0}
enum MatrixStatus matrix_init(struct Matrix *matrix, size_t size);

void matrix_destroy(struct Matrix *matrix);

const char *matrix_status_string(enum MatrixStatus status);

#endif
```

Листинг A.3 – Файл src/matrix.c

```
#include "matrix.h"

#include <stdlib.h>

enum MatrixStatus matrix_init(struct Matrix *matrix, const size_t size) {
    if (matrix == NULL || matrix->data != NULL || matrix->size != 0) {
        return MATRIX_INVALID_ARGUMENT;
    }

    if (size == 0) {
        return MATRIX_INVALID_SIZE;
    }

    if (size > SIZE_MAX / size) {
        return MATRIX_SIZE_OVERFLOW;
    }

    const size_t count = size * size;
    if (count > SIZE_MAX / sizeof(*matrix->data)) {
        return MATRIX_SIZE_OVERFLOW;
    }

    int64_t *data = malloc(count * sizeof(*data));
    if (data == NULL) {
        return MATRIX_NO_MEMORY;
    }

    matrix->size = size;
    matrix->data = data;

    return MATRIX_OK;
}
```

```
void matrix_destroy(struct Matrix *matrix) {
    if (matrix == NULL) {
        return;
    }

    free(matrix->data);

    matrix->data = NULL;
    matrix->size = 0;
}

const char *matrix_status_string(const enum MatrixStatus status) {
    switch (status) {
        case MATRIX_OK:
            return "success";
        case MATRIX_INVALID_ARGUMENT:
            return "invalid argument or nonempty output matrix";
        case MATRIX_INVALID_SIZE:
            return "matrix size must be a positive decimal integer";
        case MATRIX_SIZE_OVERFLOW:
            return "matrix size exceeds the supported range";
        case MATRIX_NO_MEMORY:
            return "cannot allocate matrix memory";
        case MATRIX_INVALID_ELEMENT:
            return "invalid int64 matrix element";
        case MATRIX_MISSING_ELEMENT:
            return "not enough matrix elements";
        case MATRIX_EXTRA_DATA:
            return "extra data after matrix elements";
        case MATRIX_READ_ERROR:
            return "input read error";
    }

    return "unknown matrix status";
}
```

Листинг А.4 – Файл src/matrix_txt.h

```
#ifndef LABA3_MATRIX_TXT_H
#define LABA3_MATRIX_TXT_H

#include "matrix.h"

#include <stdio.h>

enum MatrixStatus matrix_read_txt(FILE *input, struct Matrix *matrix);

#endif
```

Листинг A.5 – Файл src/matrix_txt.c

```
#include "matrix_txt.h"

enum MatrixStatus matrix_read_txt(FILE *input, struct Matrix *matrix) {
    if (input == NULL || matrix == NULL) {
        return MATRIX_INVALID_ARGUMENT;
    }
    if (matrix->data != NULL || matrix->size != 0) {
        return MATRIX_INVALID_ARGUMENT;
    }

    int64_t size;
    if (fscanf(input, "%ld", &size) != 1) {
        return ferror(input) ? MATRIX_READ_ERROR : MATRIX_INVALID_SIZE;
    }
    if (size <= 0) {
        return MATRIX_INVALID_SIZE;
    }

    struct Matrix temporary = {0};
    enum MatrixStatus status = matrix_init(&temporary, (size_t) size);
    if (status != MATRIX_OK) {
        return status;
    }

    const size_t count = temporary.size * temporary.size;
    for (size_t index = 0; index < count; index++) {
        int result = fscanf(input, "%ld", &temporary.data[index]);
        if (result != 1) {
            matrix_destroy(&temporary);
            if (ferror(input)) {
                return MATRIX_READ_ERROR;
            }
            return result == EOF ? MATRIX_MISSING_ELEMENT :
            ↪  MATRIX_INVALID_ELEMENT;
        }
    }

    char extra;
    int result = fscanf(input, " %c", &extra);
    if (ferror(input)) {
        matrix_destroy(&temporary);
        return MATRIX_READ_ERROR;
    }
    if (result == 1) {
        matrix_destroy(&temporary);
        return MATRIX_EXTRA_DATA;
    }

    *matrix = temporary;
    return MATRIX_OK;
```

```
}
```

Листинг А.6 – Файл src/permutation.h

```
#ifndef LABA3_PERMUTATION_H
#define LABA3_PERMUTATION_H

#include <stdbool.h>
#include <stddef.h>

bool permutation_next(size_t *values, size_t count, bool *negative);

#endif
```

Листинг А.7 – Файл src/permutation.c

```
#include "permutation.h"

static void exchange(size_t *a, size_t *b) {
    const size_t temp = *a;
    *a = *b;
    *b = temp;
}

bool permutation_next(size_t *values, size_t count, bool *negative) {
    if (count < 2) return false;

    size_t suffix = count - 1;
    while (suffix > 0 && values[suffix - 1] >= values[suffix]) {
        suffix--;
    }

    if (suffix == 0) return false;

    size_t pivot = suffix - 1;

    size_t replacement = count - 1;
    while (values[replacement] <= values[pivot]) {
        replacement--;
    }

    exchange(&values[pivot], &values[replacement]);
    *negative = !(*negative);

    size_t left = suffix;
    size_t right = count - 1;

    while (left < right) {
        exchange(&values[left], &values[right]);
        *negative = !(*negative);
        left++;
```

```
        right--;
    }

    return true;
}
```

Листинг A.8 – Файл src/determinant.h

```
#ifndef LABA3_DETERMINANT_H
#define LABA3_DETERMINANT_H

#include "matrix.h"

#include <stdbool.h>

struct Calculation {
    int64_t determinant;
    size_t threads;
};

// The thread limit includes the caller. Matrix must be initialized and filled
bool determinant_compute(const struct Matrix *matrix, size_t thread_limit,
↪  struct Calculation *result);

#endif
```

Листинг A.9 – Файл src/determinant.c

```
#include "determinant.h"
#include "permutation.h"

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Worker {
    const struct Matrix *matrix;
    size_t index;
    size_t thread_count;
    size_t job_count;
    int64_t *sums;
    bool success;
};

static bool prepare_permutation(size_t *permutation, size_t size, size_t job)
↪  {
    if (size == 1) {
        permutation[0] = 0;
        return false;
    }
```

```
    permutation[0] = job / (size - 1);
    permutation[1] = job % (size - 1);
    if (permutation[1] >= permutation[0]) {
        permutation[1]++;
    }

    size_t position = 2;
    for (size_t column = 0; column < size; column++) {
        if (column != permutation[0] && column != permutation[1]) {
            permutation[position++] = column;
        }
    }

    // Each out-of-order pair changes the sign of the permutation
    bool negative = false;
    for (size_t left = 0; left < size; left++) {
        for (size_t right = left + 1; right < size; right++) {
            if (permutation[left] > permutation[right]) {
                negative = !negative;
            }
        }
    }
    return negative;
}

static void *worker_run(void *argument) {
    struct Worker *worker = argument;
    const size_t size = worker->matrix->size;
    const size_t prefix = size == 1 ? 1 : 2;
    size_t *permutation = malloc(size * sizeof(*permutation));
    if (permutation == NULL) {
        perror("Cannot allocate permutation");
        worker->success = false;
        return NULL;
    }

    for (size_t job = worker->index; job < worker->job_count &&
    ↪  worker->success; job += worker->thread_count) {
        bool negative = prepare_permutation(permutation, size, job);
        int64_t sum = 0;

        do {
            int64_t product = 1;
            for (size_t row = 0; row < size; row++) {
                int64_t element = worker->matrix->data[row * size +
                ↪  permutation[row]];
                if (__builtin_mul_overflow(product, element, &product)) {
                    worker->success = false;
                    break;
                }
            }
```

```
            if (!worker->success) {
                break;
            }

            bool overflow = negative
                                ? __builtin_sub_overflow(sum, product, &sum)
                                : __builtin_add_overflow(sum, product, &sum);
            if (overflow) {
                worker->success = false;
                break;
            }
        } while (permutation_next(permutation + prefix, size - prefix,
        ↪  &negative));

        if (worker->success) {
            worker->sums[job] = sum;
        }
    }

    if (!worker->success) {
        fprintf(stderr, "Integer overflow during calculation\n");
    }
    free(permutation);
    return NULL;
}

bool determinant_compute(const struct Matrix *matrix, size_t thread_limit,
↪  struct Calculation *result) {
    if (matrix == NULL || matrix->data == NULL || matrix->size == 0 ||
    ↪  thread_limit == 0 || result == NULL) {
        fprintf(stderr, "Invalid calculation arguments\n");
        return false;
    }

    const size_t size = matrix->size;
    const size_t jobs = size == 1 ? 1 : size * (size - 1); // count amount of
    ↪  jobs
    const size_t count = thread_limit < jobs ? thread_limit : jobs; // min

    if (jobs > SIZE_MAX / sizeof(int64_t) || count > SIZE_MAX / sizeof(struct
    ↪  Worker) || count - 1 > SIZE_MAX / sizeof(
            pthread_t) || size > SIZE_MAX / sizeof(size_t)) {
        fprintf(stderr, "Calculation allocation size overflow\n");
        return false;
    }

    int64_t *sums = malloc(jobs * sizeof(*sums));
    struct Worker *workers = malloc(count * sizeof(*workers));
    pthread_t *threads = count > 1 ? malloc((count - 1) * sizeof(*threads)) :
    ↪  NULL;
    bool success = sums != NULL && workers != NULL && (count == 1 || threads
    ↪  != NULL);
```

```
if (!success) {
    fprintf(stderr, "Cannot allocate calculation memory\n");
    // free memory after this if-else
} else {
    for (size_t index = 0; index < count; index++) {
        workers[index] = (struct Worker){
            .matrix = matrix, .index = index, .thread_count = count,
            .job_count = jobs, .sums = sums, .success = true
        };
    }

    size_t created = 0;

    for (size_t index = 1; index < count; index++) {
        int code = pthread_create(&threads[created], NULL, worker_run,
        ↪  &workers[index]);

        if (code != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(code));
            success = false;
            break;
        }

        created++;
    }

    if (success) {
        worker_run(&workers[0]);
    }

    for (size_t index = 0; index < created; index++) {
        int code = pthread_join(threads[index], NULL);

        if (code != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(code));
            fflush(stderr);

            // An unjoined worker may still use the matrix and these
            ↪  arrays
            _Exit(EXIT_FAILURE);
        }
    }

    for (size_t index = 0; index < count; index++) {
        if (!workers[index].success) {
            success = false;
        }
    }

    int64_t sum = 0;
    if (success) {
```

```
            for (size_t job = 0; job < jobs; job++) {
                if (__builtin_add_overflow(sum, sums[job], &sum)) {
                    fprintf(stderr, "Integer overflow while adding partial
                    ↪  sums\n");
                    success = false;
                    break;
                }
            }
        }

        if (success) {
            *result = (struct Calculation){.determinant = sum, .threads =
            ↪  count};
        }
    }

    free(threads);
    free(workers);
    free(sums);

    return success;
}
```

Листинг A.10 – Файл src/CMakeLists.txt

```
cmake_minimum_required(VERSION 4.3)
project(laba3 C)

set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

set(THREADS_PREFER_PTHREAD_FLAG TRUE)
find_package(Threads REQUIRED)

add_executable(determinant main.c
        permutation.h
        permutation.c
        matrix.h
        matrix.c
        matrix_txt.h
        matrix_txt.c
        determinant.h
        determinant.c)

target_compile_definitions(determinant PRIVATE _POSIX_C_SOURCE=200809L)

target_compile_options(determinant PRIVATE
        -Wall
        -Wextra
        -Wpedantic
```

```
        -Wconversion
        -Wsign-conversion
        -Wshadow
        -Wformat=2
        -Wstrict-prototypes
        -Wmissing-prototypes
        -Werror
)

target_link_libraries(determinant PRIVATE Threads::Threads)
```

Листинг A.11 – Файл report/Dockerfile

```
FROM texlive/texlive:latest

WORKDIR /work/report

CMD ["sh", "-c", "pdflatex -interaction=nonstopmode -halt-on-error report.tex
↪  && pdflatex -interaction=nonstopmode -halt-on-error report.tex"]
```

Листинг A.12 – Файл report/Makefile

```
MAIN   = report
LATEX ?= pdflatex
FLAGS  = -interaction=nonstopmode -halt-on-error
SOURCES = $(MAIN).tex preamble.tex titlepage.tex $(wildcard inc/*) \
          $(wildcard ../src/*.c) ../src/CMakeLists.txt Dockerfile Makefile

$(MAIN).pdf: $(SOURCES)
        $(LATEX) $(FLAGS) $(MAIN).tex
        $(LATEX) $(FLAGS) $(MAIN).tex

xelatex:
        $(MAKE) -B LATEX=xelatex $(MAIN).pdf

clean:
        rm -f *.aux *.log *.out *.lol *.toc *.fls *.fdb_latexmk *.synctex.gz

.PHONY: xelatex clean
```

Приложение Б. Полный вывод strace

Листинг Б.1 – Трассировка программы с тремя исполнителями

```
508983 execve("./build/determinant", ["./build/determinant", "--threads", "3",
↪  "src/examples/matrix.txt"], 0x7ffc73ff9b48 /* 88 vars */) = 0
508983 brk(NULL)                        = 0x1e789000
508983 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
508983 openat(AT_FDCWD</home/nikallow/Code/1-mai/3-os/laba3>, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) =
↪  3</etc/ld.so.cache>
508983 fstat(3</etc/ld.so.cache>, {st_mode=S_IFREG|0644, st_size=85363, ...}) = 0
508983 mmap(NULL, 85363, PROT_READ, MAP_PRIVATE, 3</etc/ld.so.cache>, 0) = 0x7fd4cba7c000
508983 close(3</etc/ld.so.cache>)       = 0
508983 openat(AT_FDCWD</home/nikallow/Code/1-mai/3-os/laba3>, "/lib64/libc.so.6", O_RDONLY|O_CLOEXEC) =
↪  3</usr/lib64/libc.so.6>
508983 read(3</usr/lib64/libc.so.6>, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\2408\0\0\0\0\0\0@
↪  \0\0\0\0\0\0\0\260\316%\0\0\0\0\0\0\0\0@\08\0\16\0@\0D\0C\0\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\
↪  0\0\0\0\0@\0\0\0\0\0\0\0\20\3\0\0\0\0\0\0\20\3\0\0\0\0\0\0\10\0\0\0\0\0\0\0\3\0\0\0\4\0\0\0`\360\3
↪  3\0\0\0\0\0`\360\33\0\0\0\0\0`\360\33\0\0\0\0\0\34\0\0\0\0\0\0\0\34\0\0\0\0\0\0\0\20\0\0\0\0\0\0\0
↪  \1\0\0\0\5\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0-\36\27\0\0\0\0\0-\36\27\0\0\0\0\0
↪  \0\20\0\0\0\0\0\0\1\0\0\0\4\0\0\0\0 \27\0\0\0\0\0\0 \27\0\0\0\0\0"..., 832) = 832
508983 pread64(3</usr/lib64/libc.so.6>,
↪  "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\0\20\3\0\0\0\0\0\0\20\3\0\0\0\0\0\1
↪  0\0\0\0\0\0\0\0\3\0\0\0\4\0\0\0`\360\33\0\0\0\0\0`\360\33\0\0\0\0\0`\360\33\0\0\0\0\0\34\0\0\0\0\0
↪  \0\0\34\0\0\0\0\0\0\0\20\0\0\0\0\0\0\0\1\0\0\0\5\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0
↪  \0\0-\36\27\0\0\0\0\0-\36\27\0\0\0\0\0\0\20\0\0\0\0\0\0\1\0\0\0\4\0\0\0\0 \27\0\0\0\0\0\0
↪  \27\0\0\0\0\0\0 \27\0\0\0\0\0P\216\7\0\0\0\0\0P\216\7\0\0\0\0\0\0\20\0\0\0\0\0\0\1\0\0\0\6\0\0\0x\
↪  273\36\0\0\0\0\0x\273\36\0\0\0\0\0x\273\36\0\0\0\0\0"..., 784, 64) = 784
508983 fstat(3</usr/lib64/libc.so.6>, {st_mode=S_IFREG|0755, st_size=2482096, ...}) = 0
508983 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fd4cba7a000
508983 pread64(3</usr/lib64/libc.so.6>,
↪  "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\0\20\3\0\0\0\0\0\0\20\3\0\0\0\0\0\1
↪  0\0\0\0\0\0\0\0\3\0\0\0\4\0\0\0`\360\33\0\0\0\0\0`\360\33\0\0\0\0\0`\360\33\0\0\0\0\0\34\0\0\0\0\0
↪  \0\0\34\0\0\0\0\0\0\0\20\0\0\0\0\0\0\0\1\0\0\0\5\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0
↪  \0\0-\36\27\0\0\0\0\0-\36\27\0\0\0\0\0\0\20\0\0\0\0\0\0\1\0\0\0\4\0\0\0\0 \27\0\0\0\0\0\0
↪  \27\0\0\0\0\0\0 \27\0\0\0\0\0P\216\7\0\0\0\0\0P\216\7\0\0\0\0\0\0\20\0\0\0\0\0\0\1\0\0\0\6\0\0\0x\
↪  273\36\0\0\0\0\0x\273\36\0\0\0\0\0x\273\36\0\0\0\0\0"..., 784, 64) = 784
508983 mmap(NULL, 2067312, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3</usr/lib64/libc.so.6>, 0)
↪  = 0x7fd4cb881000
508983 mmap(0x7fd4cb9f3000, 495616, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
↪  3</usr/lib64/libc.so.6>, 0x172000) = 0x7fd4cb9f3000
508983 mmap(0x7fd4cba6c000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
↪  3</usr/lib64/libc.so.6>, 0x1eb000) = 0x7fd4cba6c000
508983 mmap(0x7fd4cba72000, 31600, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) =
↪  0x7fd4cba72000
508983 close(3</usr/lib64/libc.so.6>)   = 0
508983 mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fd4cb87e000
508983 arch_prctl(ARCH_SET_FS, 0x7fd4cb87e740) = 0
508983 set_tid_address(0x7fd4cb87ed68)  = 508983
508983 set_robust_list(0x7fd4cb87ea20, 24) = 0
508983 rseq({cpu_id_start=0, cpu_id=RSEQ_CPU_ID_UNINITIALIZED, rseq_cs=NULL, flags=0, node_id=0,
↪  mm_cid=0, slice_ctrl={request=0, granted=0, __reserved=0}, __reserved=0}, 33, 0, 0x53053053) = 0
508983 mprotect(0x7fd4cba6c000, 16384, PROT_READ) = 0
508983 mprotect(0x403000, 4096, PROT_READ) = 0
508983 mprotect(0x7fd4cbad0000, 8192, PROT_READ) = 0
508983 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
508983 getrandom("\x7e\xef\x45\xe5\x8f\x22\x4c\xbe", 8, GRND_NONBLOCK) = 8
508983 munmap(0x7fd4cba7c000, 85363)    = 0
508983 brk(NULL)                        = 0x1e789000
508983 brk(0x1e7aa000)                  = 0x1e7aa000
508983 openat(AT_FDCWD</home/nikallow/Code/1-mai/3-os/laba3>, "src/examples/matrix.txt", O_RDONLY) =
↪  3</home/nikallow/Code/1-mai/3-os/laba3/src/examples/matrix.txt>
508983 fstat(3</home/nikallow/Code/1-mai/3-os/laba3/src/examples/matrix.txt>, {st_mode=S_IFREG|0644,
↪  st_size=21, ...}) = 0
508983 read(3</home/nikallow/Code/1-mai/3-os/laba3/src/examples/matrix.txt>, "3\n1 2 3\n4 5 6\n7 8
↪  10\n", 4096) = 21
508983 read(3</home/nikallow/Code/1-mai/3-os/laba3/src/examples/matrix.txt>, "", 4096) = 0
508983 close(3</home/nikallow/Code/1-mai/3-os/laba3/src/examples/matrix.txt>) = 0
508983 rt_sigaction(SIGRT_1, {sa_handler=0x7fd4cb8f1240, sa_mask=[],
↪  sa_flags=SA_RESTORER|SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x7fd4cb89afb0}, NULL, 8) = 0
508983 rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0
508983 mmap(NULL, 8392704, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) =
↪  0x7fd4cb07d000
```

```
508983 madvise(0x7fd4cb07d000, 4096, MADV_GUARD_INSTALL) = 0
508983 rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
508983 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fd4cb87dce8, parent_tid=0x7fd4cb87d990, exit_signal=0, stack=0x7fd4cb07d000, stack_size=0x7fff80, tls=0x7fd4cb87d6c0} => {parent_tid=[508984]}, 88) = 508984
508983 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
508983 mmap(NULL, 8392704, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7fd4ca87c000
508983 madvise(0x7fd4ca87c000, 4096, MADV_GUARD_INSTALL) = 0
508983 rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
508983 clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x7fd4cb07cce8, parent_tid=0x7fd4cb07c990, exit_signal=0, stack=0x7fd4ca87c000, stack_size=0x7fff80, tls=0x7fd4cb07c6c0} => {parent_tid=[508985]}, 88) = 508985
508983 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
508983 futex(0x7fd4cb87dce8, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 2, NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>
508985 rseq({cpu_id_start=0, cpu_id=RSEQ_CPU_ID_UNINITIALIZED, rseq_cs=NULL, flags=0, node_id=0, mm_cid=0, slice_ctrl={request=0, granted=0, __reserved=0}, __reserved=0}, 33, 0, 0x53053053) = 0
508985 set_robust_list(0x7fd4cb07c9a0, 24) = 0
508985 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
508985 mmap(NULL, 134217728, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fd4c2800000
508985 munmap(0x7fd4c2800000, 25165824) = 0
508985 munmap(0x7fd4c8000000, 41943040) = 0
508985 mprotect(0x7fd4c4000000, 135168, PROT_READ|PROT_WRITE <unfinished ...>
508984 rseq({cpu_id_start=0, cpu_id=RSEQ_CPU_ID_UNINITIALIZED, rseq_cs=NULL, flags=0, node_id=0, mm_cid=0, slice_ctrl={request=0, granted=0, __reserved=0}, __reserved=0}, 33, 0, 0x53053053 <unfinished ...>
508985 <... mprotect resumed>)          = 0
508985 rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0
508985 madvise(0x7fd4ca87c000, 8368128, MADV_DONTNEED) = 0
508985 exit(0)                          = ?
508984 <... rseq resumed>)              = 0
508984 set_robust_list(0x7fd4cb87d9a0, 24) = 0
508984 rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0
508984 rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0
508984 madvise(0x7fd4cb07d000, 8368128, MADV_DONTNEED) = 0
508984 exit(0)                          = ?
508983 <... futex resumed>)             = 0
508984 +++ exited with 0 +++
508985 +++ exited with 0 +++
508983 fstat(1</dev/pts/2<char 136:2>>, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0x2), ...}) = 0
508983 write(1</dev/pts/2<char 136:2>>, "Determinant: -3\n", 16) = 16
508983 write(1</dev/pts/2<char 136:2>>, "Threads: 3\n", 11) = 11
508983 write(1</dev/pts/2<char 136:2>>, "Time: 0.003735860 s\n", 20) = 20
508983 exit_group(0)                    = ?
508983 +++ exited with 0 +++
```

Приложение B. Исходные результаты измерений

Листинг B.1 – Ручные запуски для матрицы 13 на 13

```
./build/determinant --threads 12 src/examples/matrix-13.txt
Determinant: 7736
Threads: 12
Time: 41.237441684 s

./build/determinant --threads 8 src/examples/matrix-13.txt
Determinant: 7736
Threads: 8
Time: 45.312672544 s

./build/determinant --threads 7 src/examples/matrix-13.txt
Determinant: 7736
Threads: 7
Time: 53.069405205 s

./build/determinant --threads 6 src/examples/matrix-13.txt
Determinant: 7736
Threads: 6
Time: 43.158549481 s

./build/determinant --threads 5 src/examples/matrix-13.txt
Determinant: 7736
Threads: 5
Time: 52.815789465 s

./build/determinant --threads 4 src/examples/matrix-13.txt
Determinant: 7736
Threads: 4
Time: 62.173046865 s

./build/determinant --threads 3 src/examples/matrix-13.txt
Determinant: 7736
Threads: 3
Time: 85.636897709 s

./build/determinant --threads 2 src/examples/matrix-13.txt
Determinant: 7736
Threads: 2
Time: 123.723745589 s

./build/determinant --threads 1 src/examples/matrix-13.txt
Determinant: 7736
Threads: 1
Time: 244.303998558 s
```